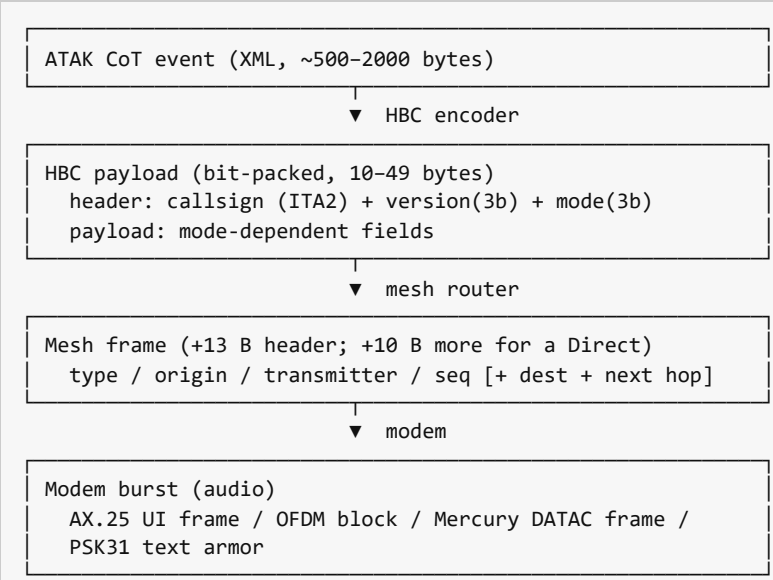
CoT → HBC Bit-Level Walkthrough

How an ATAK Cursor on Target (CoT) event becomes bits, bytes, and finally audio — every encapsulation layer, bit by bit, with a worked example for **every supported HBC mode (0–6)**.

> **Every example in this document is a real frame captured in the plugin's session debug logs during bench and field tests (Oct 2026),** then decoded back through the plugin's own JVM codec to confirm the field values shown. Nothing here is hand-invented. The source log for each example is cited, and the full list is in the Provenance appendix.

Companion documents: `PROTOCOL.md` (HBC field reference), the ICD (system-level spec), and the HBC-Protocol repository (authoritative protocol spec + Python reference implementation).

The four layers



Conventions used below

- **Bit order:** all HBC fields are packed MSB-first with no padding between fields; the finished bit stream is right-zero-padded to a byte boundary.
- **Bit streams** are shown with spaces at *field* boundaries (not byte boundaries), so you can read fields directly.
- **Coordinates:** `encoded = round(degrees × 10,000)`; latitude is a 21-bit and longitude a 22-bit two's-complement signed integer (~11 m precision).

- **Names** (marker labels, alert names, titles) are length-prefixed ASCII: a 3-bit length (0–7) followed by 8 bits per character.
- **Two ITA2 alphabets are in play** — they are *not* interchangeable:
- the **HBC header/text alphabet** (tables in `PROTOCOL.md`): 5-bit codes, CR-terminated, e.g. `K=01111`, `LTRS=11111`, `CR=01000`;
- the **mesh callsign alphabet**: the same alphabet with every 5-bit code **bit-reversed**, packed into a fixed 8-code (40-bit) field, NULL-padded — see Mesh callsign packing.

HBC header (common to all modes)

Every HBC payload starts with:

Field	Width	Notes
Callsign	variable	ITA2, 5 b/code, digits need FIGS/LTRS shifts, terminated by CR (`01000`), max 8 characters
Version	3 b	`000` = v1
Mode	3 b	`mode − 1` for Modes 1–6; Mode 0 (Ack) uses `111`

The mode table:

Mode	Wire bits	Carries	CoT types
1	`000`	PLI / Spot position	`a-f-G*`, `a-h-G*`, `a-n-G*`, `a-u-G`…
2	`001`	911 alert / cancel	`b-a-o-tbl`, `b-a-o-can`
3	`010`	GeoChat	`b-t-f`
4	`011`	Shape (circle/polygon/polyline)	`u-d-c-c`, `u-d-r`, `u-d-f`
5	`100`	CASEVAC 9-line	`b-r-f-h-c`
6	`101`	Extended marker (type+icon+tint)	any placed marker
0	`111`	Chat delivered/read receipt	`b-t-f-d`, `b-t-f-r`

Mesh encapsulation

Every HBC payload is wrapped in a mesh frame before it reaches a modem. Common 13-byte header:

Offset	Len	Field
0	1	Packet type: `0x01` Announce, `0x02` Broadcast, `0x03` Direct, `0x04` ACK
1	5	Origin callsign (bit-packed mesh ITA2)
6	5	Transmitter callsign (rewritten by each relay)
11	2	Sequence ID (big-endian)

- **Broadcast (0x02):** header + HBC payload. Used for PLIs, markers, alerts, shapes, CASEVAC — anything for everyone.
- **Direct (0x03):** header + destination (5 B) + next hop (5 B) + HBC payload. Used for chat DMs and receipts; acknowledged end-to-end.
- **Announce (0x01):** header + 1-byte hop count. No HBC payload.
- **ACK (0x04):** header + destination (5 B) + acked sequence (2 B). No HBC payload.

Mesh callsign packing (worked example)

Mesh callsign fields are 40 bits: up to 8 ITA2 codes, each the **bit-reversal** of the HBC table code, packed MSB-first and NULL-padded (`00000`). A leading LTRS is written unless the callsign would need 9 codes.

Callsign `PIX` (from the 13 Oct PSK31 test) packs to `FB 59 70 00 00`:

```
LTRS   P      I      X      NULL   NULL   NULL   NULL
11111  01101  01100  10111  00000  00000  00000  00000
└──────────────── 40 bits = FB 59 70 00 00 ─────────┘
```

(HBC codes `P=10110`, `I=00110`, `X=11101`, reversed per-code to `01101`, `01100`, `10111`.) Other callsigns appearing in the examples: `KE8TQA = F4 36 CF 87 B8`, `KE8TQB = F4 36 CF 87 B3`, `S26 = FD 37 9A 80 00`, `TAB = F8 71 30 00 00`, `PHONE = FB 4A 33 40 00`.

Announce — real frame

From `HBC_KE8TQA_OFDM_2026-10-10_12-35-28.log`:

```
12:35:31.461  DBG  TX mesh frame 14 B: 01 F4 36 CF 87 B8 F4 36 CF 87 B8 AC 45 00
12:35:31.461  LOG  Mesh: announce sent (seq AC45)
```

Bytes	Field	Value
`01`	Type	Announce
`F4 36 CF 87 B8`	Origin	KE8TQA
`F4 36 CF 87 B8`	Transmitter	KE8TQA
`AC 45`	Sequence	0xAC45
`00`	Hop count	0 (original, not a relay)

Mesh ACK — real frame

From `HBC_KE8TQB_OFDM_2026-10-09_18-23-16.log`: KE8TQB received Direct seq `5DF9` from PIX and confirms it.

```
04 F4 36 CF 87 B3 F4 36 CF 87 B3 CA BF FB 59 70 00 00 5D F9
```

Bytes	Field	Value
`04`	Type	ACK
`F4 36 CF 87 B3`	Origin	KE8TQB (the acknowledging station)
`F4 36 CF 87 B3`	Transmitter	KE8TQB
`CA BF`	Sequence	0xCABF (KE8TQB's own counter)
`FB 59 70 00 00`	Destination	PIX (origin of the Direct)
`5D F9`	Acked sequence	0x5DF9

The journey of one PLI, end to end

The flagship example: station **KE8TQA** (ATAK callsign **ONYX**) transmits its own position. From `HBC_KE8TQA_OFDM_2026-10-10_12-35-28.log`:

```
12:35:40.586  LOG  TX candidate: a-f-G-U-C uid ANDROID-8f2a5d78d919931a
12:35:40.590  DBG  TX mesh frame 27 B: 02 F4 36 CF 87 B8 F4 36 CF 87 B8 AC 46
                                       C3 2B D4 00 22 7A 72 CA C1 82 EE B3 26 C6
12:35:40.590  LOG  Queued TX mode 1 (14 B) a-f-G-U-C
12:35:42.734  LOG  TX OFDM 27 B (1840 ms burst)
```

Step 1 — ATAK hands the plugin a CoT event

ATAK's periodic self-SA event: type `a-f-G-U-C`, the device UID, a `<point lat lon>`, and a `<contact callsign="ONYX">`. (The plugin recognizes it as the *own-position* report by UID.) Roughly 1–2 kB of XML on the wire in normal TAK networking.

Step 2 — HBC encode (Mode 1): 14 bytes

```
C3 2B D4 00 22 7A 72 CA C1 82 EE B3 26 C6
```

112 bits, split at field boundaries:

```
11000 01100 10101 11101 01000   000  000   0  00  100
O     N     Y     X     CR      ver  mode  PLI aff len=4

01001111 01001110 01011001 01011000
O        N        Y        X          (name, ASCII)

00110000010111011010   11001100100110110001 10   (pad: none needed)
lat = +396218            lon = −842042 (two's complement)
```

Field	Width	Bits	Value
Callsign	25 b	`11000 01100 10101 11101 01000`	`ONYX` + CR
Version	3 b	`000`	v1
Mode	3 b	`000`	Mode 1
PLI/Spot	1 b	`0`	PLI (self position)
Affiliation	2 b	`00`	Friendly (`a-f-G`)
Name length	3 b	`100`	4
Name	32 b	ASCII	`ONYX`
Latitude	21 b	`001100000101110111010`	396218 → **39.6218°**
Longitude	22 b	`1100110010011011000110`	−842042 → **−84.2042°**

Total: 112 bits = exactly 14 bytes (no padding needed this time).

Step 3 — mesh encapsulation: 27 bytes

Broadcast frame = 13-byte header + payload:

```
02 | F4 36 CF 87 B8 | F4 36 CF 87 B8 | AC 46 | C3 2B D4 00 22 7A 72 CA C1 82 EE B3 26 C6
Bcast  origin KE8TQA   xmit KE8TQA     seq      HBC Mode 1 payload (14 B)
```

Step 4 — the modem turns 27 bytes into audio

The same mesh frame rides any of the four modems:

- **OFDM (what actually aired here):** a fixed 170-byte block — `byte 0 = 0x1B` (mesh frame length 27), bytes 1–27 = the frame, zero-padded to 170. The operator callsign also rides in the OFDM waveform's own metadata header. Polar-coded, 1500 Hz carrier — the log records the result: `TX OFDM 27 B (1840 ms burst)`.
- **AFSK/AX.25:** the frame becomes the info field of one AX.25 UI frame (dest `HBC`, source = ham callsign, control `0x03`, PID `0xF0`, HDLC CRC-16). See the Mode 4 example for a real AFSK burst line.
- **Mercury HF:** one DATAC4 frame: `byte 0 = length`, `byte 1 = 'H'` (0x48, the HBC traffic marker), bytes 2+ = the frame, zero-padded; the modem adds its CRC-16 in the last two bytes plus Mercury's exact preamble/postamble.
- **PSK31:** the binary frame is armored as text — see the PSK31 armor example below.

Step 5 — the receiver runs it backwards

Demodulate → (CRC check at the modem layer) → mesh header: not a duplicate, learn `KE8TQA = 0-hop neighbor` → HBC decode → inject CoT. What the receiving ATAK gets (reconstructed by the actual decoder from exactly these 14 payload bytes):

```
<event version="2.0" uid="HBC-ONYX" type="a-f-G" time="…" start="…"
       stale="…+5min" how="m-g" access="Undefined">
  <point lat="39.621800" lon="-84.204200" hae="9999999" ce="9999999" le="9999999"/>
  <detail>
    <contact callsign="ONYX" endpoint="*:-1:stcp"/>
    <__group name="Cyan" role="Team Member"/>
    <uid Droid="ONYX"/>
    <track speed="0.0" course="9999999.0"/>
  </detail>
</event>
```

Note what the protocol *computes instead of transmitting*: the UID (`HBC-ONYX`, derived from the callsign), timestamps (decode time), altitude/accuracy sentinels (`9999999`), and the contact endpoint that makes ONYX chat-able in ATAK.

Mode 0 — Chat delivered/read receipt

PIX read a DM from PEDRO (station S26), so PIX's ATAK generated a `b-t-f-r` read receipt; the plugin converts it into a 10-byte Mode 0 ack routed back as a mesh **Direct**. From `HBC_PIX_OFDM_2026-10-07_18-34-23.log`:

```
18:42:08.806  LOG  Mesh: direct to S26 via S26 (seq EC49)
18:42:08.807  DBG  TX mesh frame 33 B: 03 FB 59 70 00 00 FB 59 70 00 00 EC 49
                        FD 37 9A 80 00 FD 37 9A 80 00 B1 BA 81 EC 14 AB 08 4B 7F 80
18:42:08.807  LOG  Queued TX mode 0 ack (b-t-f-r) -> PEDRO tag 0x2DFE
```

Mesh Direct wrapper:

Bytes	Field	Value
`03`	Type	Direct
`FB 59 70 00 00` ×2	Origin / Transmitter	PIX
`EC 49`	Sequence	0xEC49
`FD 37 9A 80 00`	Destination	S26
`FD 37 9A 80 00`	Next hop	S26 (1 hop)

HBC payload `B1 BA 81 EC 14 AB 08 4B 7F 80` (80 bits):

```
10110 00110 11101 01000   000  111
P     I     X     CR      ver  mode 0 (wire 111)

10110 00001 01001 01010 11000 01000   01   0010110111111110   000000
P     E     D     R     O     CR     READ  tag = 0x2DFE        pad
```

Field	Width	Value
Header callsign	20 b	`PIX`
Version / Mode	3+3 b	v1, Mode 0 (`111`)
Recipient	30 b	`PEDRO` (the original DM sender's ATAK callsign)
Ack kind	2 b	`01` = READ (`b-t-f-r`); `00` = DELIVERED
Message tag	16 b	`0x2DFE` — CRC-16/CCITT-FALSE of the original ATAK messageId
Padding	6 b	zeros

On the DM sender's side, the tag is matched back to the original messageId and the familiar ATAK read checkmark appears — no UID ever crossed the air.

Mode 2 — 911 Alert (and cancel)

Station KE8TQB (ATAK callsign **FAF**) pressed the 911 button. From `HBC_KE8TQB_OFDM_2026-10-09_18-23-16.log`:

```
18:30:26.465  LOG  TX candidate: b-a-o-tbl uid ANDROID-…-9-1-1
18:30:26.467  DBG  TX mesh frame 33 B: 02 F4 36 CF 87 B3 F4 36 CF 87 B3 CA B9
                        68 DA 80 7D 19 05 18 B5 05 B1 95 A3 20 A3 18 2E EB 32 6C 50
18:30:26.467  LOG  Queued TX mode 2 (20 B) b-a-o-tbl
18:30:26.807  LOG  Ring: emergency TX (1 frame, preempt)       ← alerts may jump the turn order
```

HBC payload (20 B / 160 bits), split at field boundaries:

```
01101 00011 01101 01000   000  001   1    111
F     A     F     CR      ver  mode2 ACTIVE len=7

01000110 01000001 01000110 00101101 01000001 01101100 01100101
F        A        F         -        A        l        e          ("FAF-Ale")

011  01000110 01000001 01000110        ("FAF")
len=3  F        A        F

00110000010111011010    11001100100110110000101    0000
lat = +396218 → 39.6218°  lon = −842043 → −84.2043°  pad
```

Field	Width	Value
Callsign + ver + mode	26 b	`FAF`, v1, Mode 2
Active flag	1 b	`1` = alert active
Alert name	3 b + 56 b	`FAF-Ale` (ATAK's alert contact, truncated to 7 chars)
Originator	3 b + 24 b	`FAF`
Lat / Lon	21 + 22 b	39.6218, −84.2043

Decoded on RX: type `b-a-o-tbl`, UID `HBC-FAF-911`, `<emergency type="911 Alert"/>` — the alert updates in place on every receiving map.

The cancel (`b-a-o-can`, logged 10 s later, 13-byte payload `68 DA 80 41 A3 20 A3 18 2E EB 32 6C 50`) is the same layout with `Active = 0` and an **empty alert-name field** (`len=0`, no characters) — the originator and position still ride along, and the decoder emits `<emergency cancel="true">FAF</emergency>` against the same `HBC-FAF-911` UID.

Mode 3 — GeoChat direct message

Station PIX (ATAK callsign **PHONE2**) DMs "ROGER" to **PEDRO**. From `HBC_PIX_OFDM_2026-10-07_18-34-23.log`:

```
18:41:20.503  LOG  Mesh: direct to S26 via S26 (seq EC44)
18:41:20.504  DBG  TX mesh frame 40 B: 03 FB 59 70 00 00 FB 59 70 00 00 EC 44
            FD 37 9A 80 00 FD 37 9A 80 00 B5 30 C0 EE 7F 40 55 82 95 61 07 FD EA C6 82 A4 00
18:41:20.504  LOG  Queued TX mode 3 (17 B) b-t-f
```

The mesh Direct wrapper is identical in shape to the Mode 0 example (dest/next hop = S26). HBC payload (17 B / 136 bits):

```
10110 10100 11000 01100 00001 11011 10011 11111 01000   000  010
P     H     O     N     E     FIGS  2     LTRS  CR      ver  mode3

10   10110 00001 01001 01010 11000 01000   0011111111101111
DM   P     E     D     R     O     CR      tag = 0x3FEF

01010 11000 11010 00001 01010 01000   0000000
R     O     G     E     R     CR      pad
```

Field	Width	Value
Sender callsign	45 b	`PHONE2` (digit needs FIGS, and LTRS before CR)
Version / Mode	3+3 b	v1, Mode 3
Destination kind	2 b	`10` = Direct Message (`00` all rooms, `01` named room)
Recipient	30 b	`PEDRO` (ITA2, CR-terminated)
Message tag	16 b	`0x3FEF` (echoed later by Mode 0 receipts)
Message	30 b	`ROGER` (ITA2, CR-terminated, uppercase)
Padding	7 b	zeros

A 17-byte payload carrying sender, recipient, receipt tag and text. On decode the DM lands in the right 1:1 chat window via the derived `HBC-PEDRO` / `HBC-PHONE2` UIDs.

Mode 4 — Shape (circle)

Station PHONE (ATAK callsign **FAF**) drew a 6 m circle — and this one aired on **AFSK**, so the log also shows the AX.25 wrapper. From `HBC_PHONE_AFSK1200_2026-10-02_13-09-46.log`:

```
13:11:22.297  DBG  TX mesh frame 32 B: 02 FB 4A 33 40 00 FB 4A 33 40 00 DB E6
                       68 DA 80 CE 88 E4 C2 EE D2 DC CE 60 BB 7C C9 B1 00 01 80
13:11:22.297  LOG  Queued TX mode 4 (19 B) u-d-c-c
13:11:24.273  LOG  TX
[PHONE>HBC:\x02\xfbJ3@\x00\xfbJ3@\x00\xdb\xe6h\xda\x80\xce\x88\xe4\xc2\xee\xd2\xdc\xce`\xbb|\xc9\xb
1\x00\x01\x80] (1049 ms burst)
```

That `TX [PHONE>HBC:…]` line **is the AX.25 layer**: destination `HBC`, source `PHONE` (the legal station ID), and the info field is byte-for- byte the 32-byte mesh frame (`\x02\xfbJ3@\x00…`). The AX.25 modem adds control `0x03` (UI), PID `0xF0`, the HDLC flag preamble and CRC-16.

HBC payload (19 B / 152 bits):

```
01101 00011 01101 01000   000  011   00      111
F     A     F     CR      ver  mode4 circle  len=7

01000100 01110010 01100001 01110111 01101001 01101110 01100111
D        r        a        w        i        n        g

00110000010111011 0111     1100110010011011000100     0000000000000110   000000
lat = 396215 → 39.6215°    lon = −842044 → −84.2044°   radius = 6 m       pad
```

Field	Width	Value
Callsign + ver + mode	26 b	`FAF`, v1, Mode 4
Shape kind	2 b	`00` = circle (`01` closed polygon, `10` open polyline)
Name	3 b + 56 b	`Drawing`
Center lat / lon	21 + 22 b	39.6215, −84.2044
Radius	16 b	6 m (0–65535 m)

Polygons/polylines (`u-d-r` / `u-d-f`) replace the radius with: a 4-bit point count (2–15), the first vertex as full 21+22-bit coordinates, then each further vertex as **14+14-bit signed deltas** (±0.8191°) from the previous one — the biggest reason a 10-corner drawing still fits in a few dozen bytes.

Mode 5 — CASEVAC

Station KE8TQB (FAF) sent a CASEVAC titled `MED.9.1`. From `HBC_KE8TQB_OFDM_2026-10-09_18-23-16.log`:

```
18:30:46.194  DBG  TX mesh frame 36 B: 02 F4 36 CF 87 B3 F4 36 CF 87 B3 CA BB
                  68 DA 81 3A 6A 2A 21 71 C9 71 88 00 00 00 00 00 30 C2 C6 18 65 EC 32
18:30:46.194  LOG  Queued TX mode 5 (23 B) b-r-f-h-c
```

HBC payload (23 B / 184 bits):

```
01101 00011 01101 01000  000  100   111
F     A     F     CR     ver  mode5 len=7

01001101 01000101 01000100 00101110 00111001 00101110 00110001
M        E        D        .        9        .        1

0000000000000000   0000 0000 0000 0000 0000   0000 0000
freq = 0.00 MHz    urgent usurg prio routine conven   litter ambul

0110    00        011          00
flags   security  HLZ marking  zone protection
(casevac=0, equipment_none=1, terrain_none=1)

00101100011000011000     11001011111011000011001     0
lat = 363568 → 36.3568°    lon = −854503 → −85.4503°   pad
```

Field	Width	Value
Callsign + ver + mode	26 b	`FAF`, v1, Mode 5
Title	3 b + 56 b	`MED.9.1`
Frequency	16 b	MHz × 100 (0 here)
Patient counts	7 × 4 b	urgent, urgent-surgical, priority, routine, convenience, litter, ambulatory (all 0 here)
Flags	4 b	bit3 casevac, bit2 equipment_none, bit1 terrain_none (`0110`)
Security / HLZ / Zone	2+3+2 b	0 / 3 / 0
Lat / Lon	21 + 22 b	36.3568, −85.4503

The decoder rebuilds a `b-r-f-h-c` event with a `<_medevac_>` detail block carrying the 9-line summary fields.

Mode 6 — Extended marker

Station KE8TQA (ONYX) placed a **hostile ground marker** named `R.10.12`. Mode 6 is what lets a placed marker keep its exact CoT type, icon and color across the radio link. From `HBC_KE8TQA_OFDM_2026-10-10_12-38-16.log`:

```
12:38:34.276  LOG  TX candidate: a-h-G uid 2395f9d0-8647-4eb2-9ae7-e8be9604a87d
12:38:34.277  DBG  TX mesh frame 37 B: 02 F4 36 CF 87 B8 F4 36 CF 87 B8 5B F0
         C3 2B D4 0B D4 8B 8C 4C 0B 8C 4C 8C 16 F9 99 4F 19 C9 5A 0F FF FF FF FC
12:38:34.277  LOG  Queued TX mode 6 (24 B) a-h-G
```

HBC payload (24 B / 192 bits):

```
11000 01100 10101 11101 01000   000  101   111
O     N     Y     X     CR      ver  mode6 len=7

01010010 00101110 00110001 00110000 00101110 00110001 00110010
R        .        1        0        .        1        2

00110000101101111100     11001100101001111100011
lat = 396156 → 39.6156°    lon = −841245 → −84.1245°

0011   100100 101011 010000    01       1     11111111111111111111111111111111   00
count  'a'=36 'h'=43 'G'=16    2525C    tint  tint ARGB = 0xFFFFFFFF (white)      pad
```

Field	Width	Value
Callsign + ver + mode	31 b	`ONYX`, v1, Mode 6
Name	3 b + 56 b	`R.10.12`
Lat / Lon	21 + 22 b	39.6156, −84.1245
Token count	4 b	3 — the CoT type is sent as per-dash tokens
Type tokens	3 × 6 b	indices into `0-9 A-Z a-z`: `a`(36), `h`(43), `G`(16) → **`a-h-G`**
Icon kind	2 b	`01` = MIL-STD-2525C (icon path derivable from the type)
Tint flag + ARGB	1 + 32 b	white (0xFFFFFFFF)

Icon kinds `10` (spot-map: 4-bit palette index, or 15 + raw 32-bit ARGB) and `11` (custom iconset: 128-bit UUID + ITA2 sub-path) carry the other marker families. The decoder rebuilds `<usericon iconsetpath="COT_MAPPING_2525C/a-h/a-h-G"/>` — the red hostile diamond appears on every receiving map with its name and tint.

PSK31 text armor — real frame

PSK31's Varicode has no error detection, so on that modem the finished mesh frame is converted to a short text "armor" line instead of raw bytes. From the 13 Oct PSK31 bench test, PIX's announce:

```
Mesh frame (14 B):  01 FB 59 70 00 00 FB 59 70 00 00 66 B5 00
CRC-16/X.25:        0x15A8  (appended big-endian → 16 bytes total)
Base64 (unpadded):  AftZcAAA+1lwAABmtQAVqA
On the air:         ~AftZcAAA+1lwAABmtQAVqA~
```

The receiver hunts for `~…~` spans, Base64-decodes, verifies the CRC, and only then hands the frame to the mesh layer — human PSK31 chatter sharing the frequency is simply ignored, and a corrupted

frame is dropped rather than mis-delivered. At 31.25 baud this 24-character line is roughly 10 seconds of continuous carrier (the log measured the burst at 9.9 s; every other modem sends the same 14 bytes in 1–2 s — PSK31 trades speed for extreme weak-signal reach).

Size summary (the examples above)

What	CoT XML (typical)	HBC payload	On-air mesh frame
Announce	—	—	14 B
Mesh ACK	—	—	20 B
Mode 1 PLI	~1–2 kB	14 B	27 B broadcast
Mode 0 read receipt	~700 B	10 B	33 B direct
Mode 2 911 alert	~1 kB	20 B	33 B broadcast
Mode 3 chat DM ("ROGER")	~900 B	17 B	40 B direct
Mode 4 circle	~1.2 kB	19 B	32 B broadcast
Mode 5 CASEVAC	~1.5 kB	23 B	36 B broadcast
Mode 6 hostile marker	~1.2 kB	24 B	37 B broadcast

Provenance — the logs these frames came from

All frames were captured by the plugin's own session debug logger (`DBG TX mesh frame …` lines) during October 2026 bench/field testing, and re-decoded with the plugin's JVM codec (`codec-test/`) to produce the decoded values and CoT reconstructions shown:

- `HBC_KE8TQA_OFDM_2026-10-10_12-35-28.log` — announce, Mode 1 PLI
- `HBC_KE8TQA_OFDM_2026-10-10_12-38-16.log` — Mode 6 marker
- `HBC_PIX_OFDM_2026-10-07_18-34-23.log` — Mode 0 ack, Mode 3 DM
- `HBC_KE8TQB_OFDM_2026-10-09_18-23-16.log` — Mode 2 alert + cancel, Mode 5 CASEVAC, mesh ACK
- `HBC_PHONE_AFSK1200_2026-10-02_13-09-46.log` — Mode 4 circle + AX.25 burst line
- `HBC_PIX_PSK31_2026-10-13_21-59-49.log` — PSK31 announce (armor example)

HBC Radio — KE8TQB — github.com/ke8tqb/HBC-ATAK-Plugin